Building Qt for Qualcomm Snapdragon 8155P Board on Windows 10

Having Green Hills INTEGRITY and Qualcomm development environments setup is a required before proceeding. See **Installing Platform Dependencies**.

Creating a batch script for Windows 10

To set up the development environment for Qt for INTEGRITY, build Qt from the sources for the Qualcomm Snapdragon 8155P board. Before building Qt for INTEGRITY, set up build environment. Here a batch script to automate that, as it needs to be done before each build.

For Windows 10 create a new batch script *setEnvironment.bat*, and save it under your home folder. Add the following export variables to the script:

```
@echo off
set PATH=%PATH%;C:\ghs\comp_202014
set INTEGRITY_DIR=C:\Users\user\ghs_pack\es7_dev_env\hlos_dev_boot\apps\ghs_apps_proc\integrity
set INTEGRITY_BSP=platform-sa8155
set INTEGRITY_BUILD_TARGET=chk
set QC_MULTIMEDIA_INC_DIR=C:\Users\user\ghs_pack\es7_dev_env\hlos_dev_boot\apps\ghs_apps_proc\qc_bsp\include\amss\multimedia
set GL_INC_DIR=C:\Users\user\ghs_pack\es7_dev_env\hlos_dev_boot\apps\ghs_apps_proc\qc_bsp\AMSS\multimedia\graphics\include\public
set GL_LIB_DIR=C:\Users\user\ghs_pack\es7_dev_env\hlos_dev_boot\apps\ghs_apps_proc\qc_bsp\out\rel\libs\base;C:\Users\user\ghs_pack\es7_dev
set TARGET_ROOT_PATH="C:\Users\user\ghs_pack\es7_dev_env\hlos_dev_boot"
```

These export paths assume you have used the `C:\ghs\comp_202014 and C:\Users\user\ghs_pack\*` installation directories while installing the MULTI IDE and INTEGRITY (see **Installing Platform Dependencies**). If you have not used the default directories, adjust the export paths accordingly.

To initialize your build environment, run the following command in a Windows CMD terminal:

```
setEnvironment.bat
```

Note: Run this command in the Windows CMD terminal every time you build Qt, or use the `qmake` or `CMake` build systems.

Getting Qt sources

You can download the Qt source code from your **Qt Account**.

You can also get the Qt sources from the Git repo, see **Getting source code**.

Note: Qt sources version must be 6.2 or later.

Note: Qt modules supported by Qt for INTEGRITY are listed in **Supported Qt Modules**.

Building Qt Desktop for cross-compiling tools usage

The cross-compiling Qt requires a host build of Qt being available. During the build, tools such as moc, rcc, qmlcachegen, qsb, and others, are invoked from there. For more detailed information please read **Cross-compiling Qt**.

Run the following commands for Windows 10 Host build:

```
mkdir hostbuild
cd hostbuild/
C:/Users/user/qt5/configure -nomake tests -nomake examples -release -prefix C:/Users/user/hostbuild
cmake --build . --parallel 6
cmake --install .
```

Creating a toolchain file for INTEGRITY

To cross-compile a project with CMake, one must specify a toolchain file. This CMake-language file sets the right values for the platform name, the compiler/linker used, and a whole

```
# Copyright (C) 2022 The Qt Company Ltd.
# SPDX-License-Identifier: LicenseRef-Qt-Commercial OR BSD-3-Clause

#generic
set(CMAKE_SYSTEM_NAME Integrity)
set(CMAKE_SYSTEM_PROCESSOR arm)
set(CMAKE_CROSSCOMPILING True)

#path to installed GHS compiler for WIN10
set(TARGET_ROOT_PATH "C:/Users/user/ghs_pack/es7_dev_env/hlos_dev_boot")
set(CMAKE_BUILD_TYPE "Release")

set(GHS_COMPILER C:/ghs/comp_202014/cxintarm64.exe)
set(CMAKE_C_COMPILER ${GHS_COMPILER})
set(CMAKE_CXX_COMPILER ${GHS_COMPILER})
set(CMAKE_ASM_COMPILER ${GHS_COMPILER})

set(EGL_FOUND True)
set(UNIX True)

set (CMAKE_C_COMPILE_FEATURES c_std_99)
set (CMAKE_CXX_COMPILE_FEATURES
cxx_alias_templates
cxx_alignas
cxx_alignof
cxx_attributes
cxx_auto_type
cxx_constexp
cxx_decltype
cxx_delegating_constructors
cxx_explicit_conversions
cxx_extern_templates
cxx_inheriting_constructors
cxx_lambdas
cxx_noexcept
cxx_nonstatic_member_init
cxx_nullptr
cxx_override
cxx_range_for
cxx_raw_string_literals
cxx_reference_qualified_functions
cxx_rvalue_references
cxx_static_assert
cxx_std_17
cxx_thread_local
cxx_unicode_literals
cxx_uniform_initialization
cxx_unrestricted_unions
cxx_variadic_macros
cxx_variadic_templates)

if (NOT DEFINED CMAKE_FIND_ROOT_PATH_MODE_PROGRAM)
    set(CMAKE_FIND_ROOT_PATH_MODE_PROGRAM NEVER)
endif()
if (NOT DEFINED CMAKE_FIND_ROOT_PATH_MODE_LIBRARY)
    set(CMAKE_FIND_ROOT_PATH_MODE_LIBRARY ONLY)
endif()
if (NOT DEFINED CMAKE_FIND_ROOT_PATH_MODE_PACKAGE)
    set(CMAKE_FIND_ROOT_PATH_MODE_PACKAGE ONLY)
endif()
if (NOT DEFINED CMAKE_FIND_ROOT_PATH_MODE_INCLUDE)
    set(CMAKE_FIND_ROOT_PATH_MODE_INCLUDE ONLY)
endif()

set(CMAKE_FIND_ROOT_PATH ${TARGET_ROOT_PATH})

#base
set(CMAKE_C_FLAGS  "-bsp $ENV{INTEGRITY_BSP} -os_dir $ENV{INTEGRITY_DIR} -non_shared -startfile_dir=$ENV{INTEGRITY_DIR}/libs/$ENV{INTEGRIT
set(CMAKE_C_FLAGS_DEBUG "-g -Omaxdebug")
set(CMAKE_C_FLAGS_RELEASE "-Ospeed -Olink -Omax -no_uvfd")
set(CMAKE_C_FLAGS "${CMAKE_C_FLAGS} --signed_fields --diag_suppress=1,82,228,236,381,611,961,997,1795,1931,1974,3148 --c++17 --thread_loca
```

```
set(CMAKE_FIND_LIBRARY_PREFIXES "lib")
set(CMAKE_FIND_LIBRARY_SUFFIXES ".a")

set(BUILD_SHARED_LIBS OFF)
set(QT_CFLAGS_OPTIMIZE_FULL "-Ospeed -Olink -Omax -no_uvfd")

set(EGL_INCLUDE_DIR $ENV{GL_INC_DIR})
set(PKG_EGL_LIBRARY_DIRS ${TARGET_ROOT_PATH})

set(GLESv2_INCLUDE_DIR $ENV{GL_INC_DIR})
set(OPENGL_INCLUDE_DIR $ENV{GL_INC_DIR})

set(EGL_LIBRARY "${EGL_LIBRARY_GRAPHIC_PATH}/libESXEGL_Adreno.a")

set(GLESv2_INCLUDE_DIR $ENV{GL_INC_DIR})
set(GLESv2_LIBRARY "${EGL_LIBRARY_GRAPHIC_PATH}/libESXGLESv2_Adreno.a")

set(IntegrityPlatformGraphics_INCLUDE_DIR ${GL_INC_DIR})
set(IntegrityPlatformGraphics_LIBRARY "${EGL_LIBRARY_GRAPHIC_PATH}/libadreno_utils.a")
set(IntegrityPlatformGraphics_LIBRARIES_PACK
    "${EGL_LIBRARY_BASE_PATH}/libplanedef.a"
    "${EGL_LIBRARY_BASE_PATH}/libmmosalfile.a"
    "${EGL_LIBRARY_BASE_PATH}/libOSAbstraction.a"
    "${EGL_LIBRARY_OPENWFD_PATH}/libopenwfd.a"
    "${EGL_LIBRARY_GRAPHIC_PATH}/libOSUser.a"
    "${EGL_LIBRARY_GRAPHIC_PATH}/libpanel.a"
    "${EGL_LIBRARY_GRAPHIC_PATH}/libGSLUser.a"
    "${EGL_LIBRARY_PREBUILD_PATH}/libglnext-llvm.a"
    "${EGL_LIBRARY_PLATFORM_PATH}/libpmem.a"
    "${EGL_LIBRARY_CHK_PATH}/libposix.a"
    "${EGL_LIBRARY_CHK_PATH}/libivfs.a"
)

list(APPEND _qt_igy_gui_libs
    "${GLESv2_LIBRARY}"
    "${IntegrityPlatformGraphics_LIBRARY}"
    "${IntegrityPlatformGraphics_LIBRARIES_PACK}")

set(OPENGL_opengl_LIBRARY ${EGL_LIBRARY})

# Command is required to fix CMake WIN bug https://gitlab.kitware.com/cmake/cmake/-/issues/22933
set(CMAKE_CXX_COMPILER_PREDEFINES_COMMAND ${CMAKE_CXX_COMPILER})
list(APPEND CMAKE_CXX_COMPILER_PREDEFINES_COMMAND -bsp $ENV{INTEGRITY_BSP} -os_dir $ENV{INTEGRITY_DIR} -E -dM "${CMAKE_ROOT}/Modules/CMake

# Set format for creating static library
set(CMAKE_CXX_CREATE_STATIC_LIBRARY "${CMAKE_CXX_COMPILER} ${CMAKE_CXX_FLAGS} <LINK_FLAGS> -archive -o <TARGET> <OBJECTS> -keep_objs")
set(CMAKE_C_CREATE_STATIC_LIBRARY   "${CMAKE_CXX_COMPILER} ${CMAKE_C_FLAGS} <LINK_FLAGS> -archive -o <TARGET> <OBJECTS> -keep_objs")

# Set format to create executables
set(CMAKE_CXX_LINK_EXECUTABLE "${CMAKE_CXX_COMPILER} ${CMAKE_CXX_FLAGS} <CMAKE_CXX_LINK_FLAGS> <LINK_FLAGS> <OBJECTS> -o <TARGET> <LINK_LI
set(CMAKE_C_LINK_EXECUTABLE   "${CMAKE_CXX_COMPILER} ${CMAKE_C_FLAGS} <CMAKE_C_LINK_FLAGS> <LINK_FLAGS> <OBJECTS> -o <TARGET> <LINK_LIBRAR
```

Configuring Qt for INTEGRITY

Configure Qt for the Qualcomm Snapdragon 8155P board using these following commands: `Windows 10 HOST` requires follow configure line:

```
mkdir targetbuild
cd targetbuild/
cmake C:/Users/user/qt5/ -DQT_HOST_PATH=/c/Users/user/hostbuild/qtbase
-DCMAKE_TOOLCHAIN_FILE=/c/Users/user/ghs_pack/es7_dev_env/hlos_dev_boot/toolchain_integrity.cmake
-DQT_QMAKE_TARGET_MKSPEC=devices/integrity-armv8-SA8155P
-DBUILD_qtdoc=OFF -DBUILD_qttranslations=OFF -DFEATURE_printdialog=OFF --debug-trycompile -DFEATURE_dbus=OFF
-DFEATURE_dnslookup=OFF -DFEATURE_glib=OFF -DFEATURE_system_pcre2=OFF -DFEATURE_sql_mysql=OFF
-DQT_FEATURE_harfbuzz=OFF -DFEATURE_pkg_config=OFF -DUNIX=ON -DCMAKE_HOST_WIN32=ON -GNinja -DCMAKE_INSTALL_PREFIX=/c/Users/user/targetbuil
```

Note: QT_HOST_PATH variable introduced in Qt 6. When cross-compiling, this must be set to the install location of Qt for the host platform. It is used to locate tools to be run on the host (moc, rcc, androiddeployqt, and so on).

Building Qt for INTEGRITY

Build Qt by calling `cmake` in the terminal. You can run `cmake` with as many cores on your host machine as you desire. In our example we use six cores:

```
cmake --build . --parallel 6
```

Installing Qt

If you have not used the configure option `-prefix $PWD/qtbase` in Configuring Qt, run the following commands in a terminal:

```
cd <Qt installation directory>
cmake --install .
```

If you have used the configure option `-prefix $PWD/qtbase`, you can use Qt from the build directory, without running the `cmake install` command.

Qt is now configured and built for the sa8155 board.

‹ Build Qt with Windows 10 host

Building and flashing a Dynamic Download Project ›

© 2026 The Qt Company Ltd. Documentation contributions included herein are the copyrights of their respective owners. The documentation provided herein is licensed under the terms of the GNU Free Documentation License version 1.3 as published by the Free Software Foundation. Qt and respective logos are trademarks of The Qt Company Ltd. in Finland and/or other countries worldwide. All other trademarks are property of their respective owners.

QtWS26 The Qt Event of the Year: Qt World Summit 2026 Save Your Spot ›

Contact Us
Qt Group
Our Story
Brand
News
Careers
Investors
Qt Products
Software Quality Products
Licensing
License Agreement
Open Source
Plans and pricing
Download
FAQ
Learn Qt
For Learners
For Students and Teachers
Qt Documentation
Qt Forum
Support & Services
Professional Services
Customer Success
Support Services
Partners
Qt World
© 2026 The Qt Company
Feedback
Qt Group includes The Qt Company Oy and its global subsidiaries and affiliates.